CODELANGID: A CHARACTER-LEVEL CNN FOR PROGRAMMING LANGUAGE IDENTIFICATION

Mohamad Salman | `mohamad.salman@mail.utoronto.ca` | **Solo project**
Code: `https://github.com/mohamadmsalman82/codelangid`

1 BRIEF PROJECT DESCRIPTION

Syntax highlighters, code search engines, snippet managers and Q&A sites like Stack Overflow all need to know what language a piece of code is written in—usually from a short fragment with no filename, because the filename is exactly what is lost when code is pasted into a chat or scraped from a page (Alreshedy et al., 2018). **This project takes a raw snippet and predicts which of 10 popular languages it is (Python, Java, C++, JavaScript, Go, C, Ruby, PHP, Rust, C#), using nothing but the characters.** Rule-based detectors do badly on short fragments (van Dam & Zaytsev, 2016) and GitHub's Linguist mostly just reads the extension (GitHub, 2025), so a learned model fills a real gap.

Deep learning suits this input and output for three reasons. What separates one language from another is many small overlapping hints—keywords, operator spellings, bracket and indentation habits—and a character-level CNN learns those itself instead of me hand-writing rules for 10 languages (Zhang et al., 2015). Those hints are short character patterns like `:=`, `fn` and `$var`, which is exactly what a small-kernel convolution spots (Kim, 2014). And reading characters avoids a circular problem: a word-based model needs a tokenizer, but you cannot pick one until you know the language.

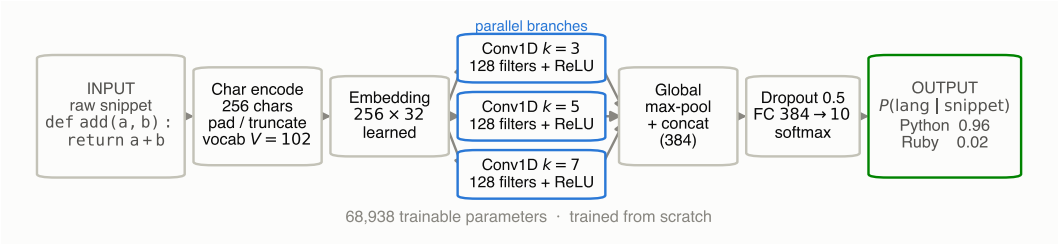

Figure 1: The CodeLangID pipeline. **Input**: a raw snippet, encoded character-by-character into a fixed 256-character window. **Output**: a probability for each of the 10 languages.

2 INDIVIDUAL CONTRIBUTIONS AND RESPONSIBILITIES

This is a **solo project**. There are no other team members, so every item in Table 1 is my own work and every responsibility is mine. All the code is in `src/` with a fixed seed (360), and every number here regenerates from a fresh checkout—except the CNN's, which moves a little from run to run (§3.4).

Table 1: Work breakdown; every item is mine alone (solo project). Only final tuning and the report remain in progress.

Component (all complete)	Evidence / artefact
Data collection & cleaning	`build_dataset.py`: 10,360 snippets (Table 2)
Held-out test collection	`scrape_heldout.py`: 740 snippets, 24 repos, 0 leaked
Baseline model	`baseline.py`: 90.14% test (Table 3)
Primary model (char-CNN)	`cnn.py`: 92.74% test, 82.84% held-out
Alternative model (BiGRU)	`gru.py`: tried and **rejected** (§3.3)
Evaluation, ablation & errors	`figures.py`, `leakage_experiment.py`: §3.4

3 NOTABLE CONTRIBUTION

My notable contribution is the **whole working pipeline** below: a cleaned dataset, a baseline, a char-CNN that beats it, and a test set I collected myself that showed exactly why the model fails.

3.1 DATA PROCESSING

Source. Rosetta Code (Rosetta Code, 2026) (via the `acmeism/RosettaCodeData` mirror) solves the same tasks in many languages, and each solution sits in a folder named after its lan-

guage, so the labels come free. Table 2 lists every cleaning step and what it removed, in enough detail for a classmate to redo (`src/build_dataset.py`, seed 360). Two steps matter most: *removing near-duplicates*, since Rosetta often stores several almost-identical versions of one solution, and *splitting by task rather than by file*. I save each snippet twice—`raw`, and `nocomment` with comments stripped (10,271 clear the 64-character floor)—to see whether the model reads the code or the comments.

Table 2: Cleaning pipeline and yield (step 2's three filters run together as each file is windowed, so the running total is first well-defined at its end). Result: 10,360 snippets, 1,036 per class (chance = 10%), from 1,162/249/250 *tasks*.

#	Operation	Removed	Remaining
1	Read 10 language directories	—	18,654 files
2	Cut into $\leq$3 windows/file of 256 chars ($\geq$3 lines, $\geq$64 chars); drop windows $>$50% comment; drop exact duplicates (SHA-1 of normalised text)	1,125 files, 3,787 win.	38,394 snippets
3	Drop near-duplicates: MinHash over 5-char shingles (64 perms, 16 LSH bands), drop pairs overlapping $\geq$80% (Broder, 1997)	353	38,041
4	Balance: cap each class at 1,036 = smallest class (PHP)	27,681	**10,360**
5	Split *by task* 70/15/15; encode to vocab V=102, pad/truncate to 256	—	7,225 / 1,634 / 1,501

One cleaned training sample, copied exactly from `raw_train.jsonl` (PYTHON, task `Call-a-foreign-language-function`, 121 characters; the two # comments are what `nocomment` removes):

```
import ctypes
libc = ctypes.CDLL("/lib/libc.so.6")
libc.strcmp("abc", "def") # -1
libc.strcmp("hello", "hello") # 0
```

Plan for never-before-seen data. Instead of reusing a public dataset I collected the final test set myself (`src/scrape_heldout.py`): **740 snippets from 24 real GitHub repositories**, 74 per language, cleaned the same way. None links to Rosetta Code, and it is deliberately a different *kind* of code—real library code, not puzzle answers. I checked every snippet against every training snippet: **none overlap**, and the model never trains on it.

Challenges. (i) Rosetta has little PHP (645 files vs. Python's 3,054), and since I keep the classes balanced, that shortage drags *every* class down to 1,036—the main limit on dataset size. (ii) I split by task assuming a per-snippet split would let the model memorise the same solution twice. I measured it (`src/leakage_experiment.py`) and I was wrong: 96.9% of a per-snippet test set does share a task with training, yet the deterministic baseline gains only $+0.20$ points from it, and the CNN's gain sits inside its own run-to-run noise (§3.4). Near-duplicate removal had already closed the gap. I kept the by-task split as the safer choice. (iii) My comment remover works line by line, so it wrongly deletes text inside strings containing `//` or #, and in C deletes `#include`/`#define`, which are real code. That hurts the no-comment score rather than helping, so the 92.49 below is if anything too low and my conclusion is safe. Fixed for the final.

3.2 BASELINE MODEL

The baseline uses no deep learning: it counts character sequences 1 to 3 long, weights them by TF-IDF, and feeds them to **Multinomial Naive Bayes** on the same splits (Khasnabish et al., 2014; Ugurel et al., 2002). Nothing to tune beyond that 1–3 choice, 43,573 features, 4 seconds to train. It is a real bar: **90.14%**, where guessing gets 10%. Simple character counting already does most of the job, so the CNN has to earn the rest.

Qualitative (baseline). Its GitHub mistakes are about English, not code. It calls PHP C# **34 times out of 74** and gets PHP right only **26**, though it never mixes those two up on Rosetta. Java is called Python 24 times and C is called Python 20 times, for the same reason: when a snippet is mostly English comment, a word-counting model drifts toward whichever language had the most English in training. **Challenge:** no tuning fixes this—the problem is what the model looks at, not its settings. That is the gap the CNN closes.

3.3 PRIMARY MODEL

The main model (Figure 1): `Embedding`$(102, 32)$ turns each character into 32 numbers; three convolutions run *side by side* over 3, 5 and 7 characters (128 filters each, then ReLU); each keeps only its strongest response (global max-pool); the three join into 384 numbers; `Dropout`(0.5) drops half during training; `Linear`$(384, 10)$ plus softmax gives the answer. Three widths side by side, rather than one deep stack, let it see 3-, 5- and 7-character clues at once. It has **68,938 trainable parameters** across 5 weight layers, trained from scratch with cross-entropy loss and Adam (Kingma & Ba, 2015) (learning rate 10^{-3}, batch 64, at most 30 epochs, stopping once validation accuracy has not improved for 6), in **35 seconds** on an Apple M-series GPU. Its training and validation accuracy

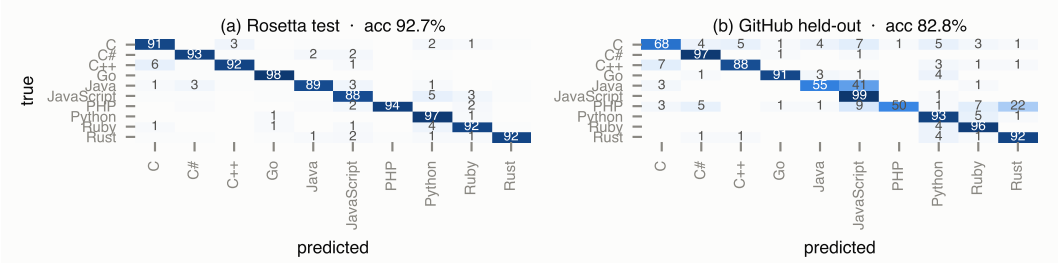

Figure 2: Char-CNN confusion (each cell = % of that true class predicted as that column). (a) Rosetta test. (b) The never-before-seen GitHub set, where Java→JavaScript and PHP→Rust dominate (§3.4).

curves drift apart as it trains—0.7 points at epoch 10, 4.7 by epoch 25—so it is slowly memorising. Early stopping on validation accuracy caught that: it halted at 25 and kept **epoch 19**.

I also built the bidirectional GRU I promised in the proposal (`src/gru.py`; 42,186 parameters). **It did not work out**, and it is the most useful thing I have learned so far: 90.27% on test, no better than Naive Bayes, for **647 seconds of training, 18 times the CNN**. What gives a language away in 256 characters are short local patterns, so a model built to track long-range order pays for something this task never uses.

3.4 Quantitative and Qualitative Results

Quantitative. The char-CNN is the best model on every split (Table 3): **92.74%** on the Rosetta test set (macro-F1 0.927, loss 0.238) against the baseline's 90.14, and **82.84%** on the never-before-seen GitHub set (macro-F1 0.822, loss 0.495) against 79.59 —so it wins by **2.6 points** in-distribution and **3.2** out of it.

Table 3: All models on both test sets (accuracy %; chance 10%). Best test / held-out in bold; `ft` = n-gram features, not learned weights. The CNN rows are one committed run; see Challenges for how much that number moves between runs.

Model	Variant	Params	Val	Test (Rosetta)	Held-out (GitHub)
TF-IDF + NB (baseline)	raw	43,573 ft	91.80	90.14	79.59
TF-IDF + NB (baseline)	nocomment	41,472 ft	91.48	89.27	81.89
Char-BiGRU (rejected)	raw	42,186	92.59	90.27	80.54
Char-CNN (primary)	raw	68,938	94.31	**92.74**	**82.84**
Char-CNN (primary)	nocomment	68,938	94.69	92.49	82.16

What happens without comments. Deleting every comment costs the CNN just **0.25** (92.74 to 92.49), so it reads the *structure of the code*, not the English in the comments. The baseline does the opposite: take its comments away and its GitHub score *rises* 2.30 (79.59 to 81.89).

Qualitative (CNN). The model scores 9.9 points lower on GitHub than on Rosetta (Figure 2), and reading the mistakes explains why. They fall into two groups. *(1) Java called JavaScript, 30 times: licence headers.* **22 of those 30** start with an Apache or copyright header, and on average only **17% of the snippet is code**—the rest is English inside a `/* */` block. Rosetta files never have these headers, so the model never learned that a real file often opens with boilerplate that looks identical in every language. It is not really wrong; it just has almost no code to look at. *(2) PHP called Rust, 16 times: PHP 8 attributes.* **13 of those 16** contain `#[`, and the training data explains it: `#[` appears in **0% of PHP snippets (0 of 753)** but **9% of Rust (63 of 703)**, so the model sensibly learned `#[` means Rust. But modern PHP 8 writes attributes exactly the same way (`#[Group('...')]`), and they appear in **19% of my GitHub PHP (14 of 74)**. Rosetta's PHP predates PHP 8, so the model never saw it—a textbook case of test data not matching training data. PHP's F1 drops 0.97 to 0.66.

Challenges. (i) The GitHub test set took the most effort: my first attempt got only 34 snippets per language because the C repositories I chose were too small. The GRU's 18× cost also made it impractical to tune. (ii) Apple's GPU does not give identical results twice, so a fixed seed does not pin the number down. I reran the committed setup eight times and got anywhere from **91.01 to 92.74** on test (average 92.2). Table 3 reports the committed run, which is the *highest* of the eight, so 92.74 is the top of a range rather than a typical score. The ordering is what holds up: the CNN beat the baseline in **all eight**, the closest by 0.87 points. **Can I finish in time?** Yes. The data is collected, the baseline is beaten on both test sets, and the pipeline reruns in under 3 minutes. The error analysis already says what to do next: strip licence headers, add post-2015 PHP, and add the SCC dataset (Alreshedy et al., 2018).

REFERENCES

Kamel Alreshedy, Dhanush Dharmaretnam, Daniel M. German, Venkatesh Srinivasan, and T. Aaron Gulliver. SCC: Automatic classification of code snippets. In *IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pp. 203–208. IEEE, 2018. Preprint: arXiv:1809.07945.

Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, pp. 21–29. IEEE, 1997.

GitHub. GitHub linguist. `https://github.com/github-linguist/linguist`, 2025. Accessed: 2026-07-15.

Jyotiska Nath Khasnabish, Mitali Sodhi, Jayati Deshmukh, and G. Srinivasaraghavan. Detecting programming language from source code using bayesian learning techniques. In *Machine Learning and Data Mining in Pattern Recognition (MLDM)*, pp. 513–522. Springer, 2014.

Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746–1751, 2014.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Rosetta Code. Rosetta code. `https://rosettacode.org`, 2026. Corpus mirror: `https://github.com/acmeism/RosettaCodeData`. Accessed: 2026-07-15.

Secil Ugurel, Robert Krovetz, and C. Lee Giles. What's the code? automatic classification of source code archives. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 632–638, 2002.

Juriaan Kennedy van Dam and Vadim Zaytsev. Software language identification with natural language classifiers. In *IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pp. 624–628, 2016.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pp. 649–657, 2015.